Title: Multithreading and Resource Utilization

1. AIM

To study and implement multithreading using POSIX threads (Pthreads) in the C programming language. To create multiple threads within a single process, execute concurrent processing tasks, share application memory, and understand thread synchronization and resource utilization.

2. THEORY

2.1 Introduction to Multithreading

Multithreading is a technique in which multiple threads execute concurrently within a single process.

A thread is the smallest unit of CPU execution within a process. A process can contain multiple threads that perform different tasks simultaneously.

Threads belonging to the same process share resources such as:

- Program code.
- Global variables.
- Heap memory.
- Open files.
- Process resources.

However, each thread maintains its own:

- Thread ID.
- Stack.
- CPU registers.
- Program counter.

Multithreading improves application responsiveness and allows multiple tasks to execute concurrently.

2.2 Introduction to POSIX Threads (Pthreads)

POSIX Threads, commonly known as Pthreads, is a standard programming interface used to create and manage threads in Unix-like operating systems.

Pthreads provides functions to:

- Create new threads.
- Execute concurrent functions.
- Wait for thread completion.
- Synchronize shared resources.
- Control thread execution.

In C programming, Pthreads functionality is provided through the `pthread.h` header file.

2.3 Thread Creation

The `pthread_create()` function is used to create a new thread.

Syntax:

```
None
pthread_create(&thread, NULL, function_name, argument);
```

Parameters:

1. `thread`: Stores the thread identifier.
2. `NULL`: Specifies default thread attributes.
3. `function_name`: Function executed by the new thread.
4. `argument`: Argument passed to the thread function.

Example:

```
None
pthread_t thread;


pthread_create(&thread, NULL, myFunction, NULL);
```

2.4 Thread Execution

After a thread is created, it starts executing the function specified in `pthread_create()`.

Multiple threads can execute concurrently, depending on the operating system scheduler and available CPU cores.

Example:

```
None
void *myFunction(void *arg)

{

    printf("Thread is executing\n");

    return NULL;

}
```

2.5 Thread Joining

The `pthread_join()` function is used to wait for a particular thread to finish its execution.

Syntax:

```
None
pthread_join(thread, NULL);
```

It ensures that the main program waits until the specified thread completes.

Advantages:

- Synchronizes thread execution.
- Prevents premature termination of the main thread.
- Allows collection of thread return values when required.

2.6 Thread Identification

Each thread has a unique thread identifier.

The `pthread_self()` function returns the identifier of the currently executing thread.

Syntax:

```
None
pthread_t tid = pthread_self();
```

It helps identify individual threads during program execution.

2.7 Shared Memory in Multithreading

Threads within the same process share the process's memory space.

This allows multiple threads to access common variables and data structures.

For example, multiple threads can access a shared counter or an array.

Advantages of shared memory:

1. Faster communication between threads.
2. Easy sharing of application data.
3. Reduced memory overhead compared to separate processes.
4. Efficient execution of parallel tasks.

However, simultaneous modification of shared data may cause data inconsistency.

2.8 Thread Synchronization

Thread synchronization is the process of coordinating multiple threads to ensure safe access to shared resources.

When multiple threads access and modify the same variable simultaneously, a race condition may occur.

A race condition occurs when the result of a program depends on the unpredictable order of execution of multiple threads.

Synchronization techniques help prevent such problems.

2.9 Mutex (Mutual Exclusion)

A mutex is a synchronization mechanism used to protect shared resources.

Only one thread can lock a mutex and access the protected critical section at a time.

Important functions:

Function	Description
`pthread_mutex_init()`	Initializes a mutex

`pthread_mutex_lock()`	Locks a mutex
`pthread_mutex_unlock()`	Unlocks a mutex
`pthread_mutex_destroy()`	Destroys a mutex

Example:

```
None
pthread_mutex_lock(&mutex);


shared_counter++;


pthread_mutex_unlock(&mutex);
```

This ensures that only one thread modifies the shared counter at a time.

2.10 Resource Utilization

Resource utilization refers to the efficient use of system resources such as CPU, memory and processing time.

Multithreading helps improve resource utilization by allowing multiple tasks to make progress concurrently.

Benefits include:

- Better CPU utilization.
- Improved application responsiveness.
- Efficient memory sharing.
- Faster execution of independent tasks.

- Better utilization of multicore processors.

2.11 Advantages of Multithreading

1. Multiple tasks can execute concurrently.
2. Threads share the same process memory.
3. Thread creation is generally less expensive than process creation.
4. Improves application responsiveness.
5. Efficient utilization of multicore processors.
6. Supports parallel processing.

2.12 Disadvantages of Multithreading

1. Requires proper synchronization.
2. Race conditions may occur.
3. Debugging can become difficult.
4. Deadlocks may occur due to improper locking.
5. Excessive thread creation can increase system overhead.

2.13 Difference Between Process and Thread

Process	Thread
Independent execution unit	Smallest execution unit within a process
Has a separate address space	Shares process address space
Process creation is comparatively expensive	Thread creation is comparatively lightweight
Communication requires IPC mechanisms	Communication can use shared memory
Has separate process resources	Shares many process resources

3. CODE

Program: Implementation of Multithreading Using POSIX Threads in C

Filename: `multithreading.c`

This program creates multiple threads, displays their thread IDs, performs concurrent operations and demonstrates shared resource access using a mutex.

```
None
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <unistd.h>


// Shared resource

int shared_counter = 0;


// Mutex declaration

pthread_mutex_t mutex;


// Thread function

void *thread_function(void *arg)

{

    int thread_id = *(int *)arg;


    printf("\nThread %d started\n", thread_id);
```

```
printf("Thread ID: %lu\n",
       (unsigned long)pthread_self());

// Perform shared resource operation
for (int i = 0; i < 5; i++)
{
    pthread_mutex_lock(&mutex);

    shared_counter++;

    printf("Thread %d: Shared Counter = %d\n",
           thread_id, shared_counter);

    pthread_mutex_unlock(&mutex);

    sleep(1);
}

printf("Thread %d completed\n", thread_id);

return NULL;
```

```
}


int main()
{
    pthread_t threads[3];

    int thread_ids[3] = {1, 2, 3};

    // Initialize mutex
    pthread_mutex_init(&mutex, NULL);

    printf("Main Thread Started\n");

    // Create three threads
    for (int i = 0; i < 3; i++)
    {
        int result = pthread_create(
            &threads[i],
            NULL,
            thread_function,
            &thread_ids[i]
```

```
        );

        if (result != 0)
        {
            printf("Error creating thread %d\n",
                   i + 1);

            return 1;
        }
    }

    // Wait for all threads to complete
    for (int i = 0; i < 3; i++)
    {
        pthread_join(threads[i], NULL);
    }

    // Display final shared resource value
    printf("\nAll threads completed\n");

    printf("Final Shared Counter = %d\n",
```

```
            shared_counter);


    // Destroy mutex
    pthread_mutex_destroy(&mutex);


    printf("Main Thread Completed\n");


    return 0;
}
```

Explanation of the Program

1. Header Files

- `stdio.h`: Provides input and output functions.
- `stdlib.h`: Provides standard utility functions.
- `pthread.h`: Provides POSIX thread functions.
- `unistd.h`: Provides the `sleep()` function.

2. Shared Variable

```
None
int shared_counter = 0;
```

This variable is shared by all threads because they belong to the same process.

3. Mutex Declaration

```
None
pthread_mutex_t mutex;
```

The mutex protects the shared counter from simultaneous modification.

4. Thread Creation

```
None
pthread_create(&threads[i], NULL,
               thread_function, &thread_ids[i]);
```

This creates three independent threads.

5. Thread Identification

```
None
pthread_self()
```

Returns the ID of the currently executing thread.

6. Mutex Locking

```
None
pthread_mutex_lock(&mutex);


shared_counter++;


pthread_mutex_unlock(&mutex);
```

This protects the shared counter and prevents race conditions during its modification.

7. Thread Joining

```
None
pthread_join(threads[i], NULL);
```

The main thread waits until all three threads complete execution.

8. Mutex Destruction

```
None
pthread_mutex_destroy(&mutex);
```

Releases the mutex resource after all threads have completed.

4. EXECUTION INSTRUCTIONS

Requirements

- Operating System: Ubuntu Linux / Kali Linux.
- Programming Language: C.
- Compiler: GCC.
- Library: POSIX Threads (Pthreads).
- Terminal: Linux Terminal.

Step 1: Open Linux Terminal

Press:

```
None
Ctrl + Alt + T
```

Step 2: Check GCC Compiler

Execute:

```
None
gcc --version
```

If GCC is not installed:

```
None
sudo apt update

sudo apt install build-essential
```

Step 3: Create a Practical Directory

```
None
mkdir Multithreading

cd Multithreading
```

Step 4: Create the C Program

Execute:

```
None
nano multithreading.c
```

Copy and paste the complete C program provided above.

Save the file:

1. Press `Ctrl + O`.
2. Press Enter.
3. Press `Ctrl + X`.

Step 5: Compile the Program

Execute:

```
None
gcc multithreading.c -o multithreading -pthread
```

Important: The `-pthread` option is required to compile and link the program using POSIX threads.

Step 6: Execute the Program

```
None
./multithreading
```

Step 7: Observe the Output

Expected Output (thread execution order may vary):

None

```
Main Thread Started


Thread 1 started

Thread ID: 140000001

Thread 1: Shared Counter = 1


Thread 2 started

Thread ID: 140000002

Thread 2: Shared Counter = 2


Thread 3 started

Thread ID: 140000003

Thread 3: Shared Counter = 3


Thread 1: Shared Counter = 4

Thread 2: Shared Counter = 5

Thread 3: Shared Counter = 6


...


Thread 1 completed
```

```
Thread 2 completed

Thread 3 completed


All threads completed

Final Shared Counter = 15

Main Thread Completed
```

Note: Thread IDs and execution order will vary depending on the system. The final counter should be 15.

Step 8: Verify Thread Execution

You can also use the following command to monitor threads:

```
None
ps -eLf
```

To monitor threads of a particular process:

```
None
top -H
```

Press `Q` to exit the top command.

5. RESULT

The multithreading program using POSIX threads was successfully implemented in C.

The following operations were demonstrated:

1. Created multiple threads using `pthread_create()`.
2. Executed concurrent processing using thread functions.
3. Retrieved thread identification using `pthread_self()`.
4. Shared a common variable between multiple threads.

5. Implemented synchronization using mutex locks.
6. Prevented race conditions during shared counter modification.
7. Used `pthread_join()` to wait for thread completion.
8. Released synchronization resources using `pthread_mutex_destroy()`.

The final shared counter value was 15, demonstrating successful execution of all three threads.

6. CONCLUSION

In this practical, multithreading and resource utilization using POSIX threads were studied and implemented in C programming. Multiple threads were created to execute concurrent tasks within a single process. Shared memory was used to access common resources, while mutex synchronization was implemented to prevent race conditions. The `pthread_create()`, `pthread_join()` and mutex functions were used to manage thread execution. Thus, the practical provided an understanding of concurrent programming, thread synchronization and efficient resource utilization in Linux.